PRACTICAL 11

Deploy and Scale a Dockerized AI Application Using Kubernetes and Minikube

Complete step-by-step guide for Ubuntu on WSL 2
Including files, commands, troubleshooting and viva preparation

Objective: Set up a local Kubernetes cluster with Minikube, deploy an Iris Flask AI API, frontend and MongoDB, test predictions, and scale the AI Deployment from one Pod to three Pods.

Environment: Windows + Ubuntu WSL 2 + Docker Desktop + Minikube + kubectl
Application: Iris flower classification using scikit-learn, Flask and MongoDB prediction history

1. Aim and learning outcomes

Aim: To set up a local Kubernetes cluster using Minikube and deploy and scale a Dockerized AI application.

Learning outcomes: install Minikube; start a local cluster; build and load Docker images; create Kubernetes Deployments and Services; access the application; inspect resources; and scale the AI Deployment.

Architecture

Browser → Frontend Service → Frontend Pod
Browser → AI Service → AI Pod replicas
AI Pods → MongoDB Service → MongoDB Pod + PersistentVolumeClaim

Frontend: Nginx on port 80. AI API: Flask on port 5000. MongoDB: port 27017 inside the cluster. Kubernetes Services provide stable network endpoints for Pods.

Prerequisites

Install Docker Desktop on Windows and enable Settings → Resources → WSL Integration for your Ubuntu distribution. Keep Docker Desktop running.

```
docker --version
docker ps
```

If Docker is not found in Ubuntu, enable WSL Integration in Docker Desktop, apply the setting and reopen Ubuntu.

2. Install Minikube

```
sudo apt update
sudo apt install -y curl wget
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
rm minikube-linux-amd64
minikube version
```

3. Start and verify the Kubernetes cluster

```
minikube start --driver=docker --cpus=3 --memory=4096
minikube status
minikube kubectl -- get nodes
minikube kubectl -- get pods -A
```

The node should show Ready. Minikube status should show the host, kubelet and API server as Running. Initial startup may take several minutes.

Use `minikube stop` to pause and `minikube start` to resume. Do not use `minikube delete` unless you intentionally want to remove the cluster.

4. Project folder structure

The Practical 10 application is reused as the workload for this Kubernetes practical. From the main project folder, create the Kubernetes directory:

```
cd ~/docker-compose-ai
mkdir -p kubernetes
```

```
docker-compose-ai/
■■■ ai-model/
■   ■■■ app.py
■   ■■■ requirements.txt
■   ■■■ Dockerfile
■■■ frontend/
■   ■■■ index.html
■   ■■■ Dockerfile
■■■ kubernetes/
    ■■■ mongodb.yaml
    ■■■ ai-deployment.yaml
    ■■■ ai-service.yaml
    ■■■ frontend-deployment.yaml
    ■■■ frontend-service.yaml
```

4.1 AI model file: ai-model/app.py

Create or verify the AI API file. This Flask API trains a RandomForestClassifier on scikit-learn's Iris dataset, returns predictions and stores prediction history in MongoDB.

```
cd ~/docker-compose-ai/ai-model
nano app.py
```

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from pymongo import MongoClient
from datetime import datetime
import os
import time

app = Flask(__name__)
CORS(app)

MONGO_URL = os.getenv("MONGO_URL", "mongodb://mongodb:27017/")
client = MongoClient(MONGO_URL, serverSelectionTimeoutMS=5000)
db = client["iris_database"]
collection = db["predictions"]

iris = load_iris()
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(iris.data, iris.target)

@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "AI Model API is running",
                    "status": "success"})

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return jsonify({"error": "Invalid JSON"}), 400
    features = data.get("features")
    if not isinstance(features, list) or len(features) != 4:
        return jsonify({"error": "Enter exactly 4 features"}), 400
    try:
        features = [float(value) for value in features]
        prediction = int(model.predict([features])[0])
        probabilities = model.predict_proba([features])[0]
        result = {
            "prediction": str(iris.target_names[prediction]),
            "class_id": prediction,
            "confidence": round(float(max(probabilities)), 4)
        }
        collection.insert_one({
            "features": features,
            "prediction": result["prediction"],
            "confidence": result["confidence"],
            "timestamp": datetime.utcnow()
        })
        return jsonify(result)
    except (ValueError, TypeError):
        return jsonify({"error": "Features must be numeric"}), 400
    except Exception:
        return jsonify({"error": "Prediction or database error"}), 500

@app.route("/history", methods=["GET"])
def history():
    records = list(collection.find({}, {"_id": 0})
                   .sort("timestamp", -1).limit(20))
    for record in records:
        record["timestamp"] = record["timestamp"].isoformat()
    return jsonify(records)

if __name__ == "__main__":
    for attempt in range(15):
        try:
            client.admin.command("ping")
            print("Connected to MongoDB successfully")
            break
        except Exception:
            print("Waiting for MongoDB...")
            time.sleep(2)
    else:
        raise RuntimeError("Could not connect to MongoDB")
    app.run(host="0.0.0.0", port=5000)
```

Save Nano files with Ctrl+O, Enter, then Ctrl+X.

4.2 Python dependencies: ai-model/requirements.txt

```
cd ~/docker-compose-ai/ai-model
nano requirements.txt
```

```
Flask
flask-cors
scikit-learn
pymongo
```

4.3 AI Dockerfile: ai-model/Dockerfile

```
nano Dockerfile
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --timeout=300 --retries=10 -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

4.4 Frontend: frontend/index.html and frontend/Dockerfile

Reuse the working index.html from Practical 10. It must make browser-side requests to http://localhost:5000/predict (POST JSON with four numeric features) and http://localhost:5000/history (GET). The browser is on the host machine, so localhost:5000 is the AI port-forward address.

```
cd ~/docker-compose-ai/frontend
nano Dockerfile
```

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

If index.html is missing, recreate the Practical 10 frontend: four numeric Iris inputs, a Predict button calling POST /predict, and a View History button calling GET /history. The Dockerfile above packages that static page with Nginx.

5. Build Docker images

Run the build commands from the main project directory, not from the kubernetes subdirectory.

```
cd ~/docker-compose-ai
docker build -t iris-ai-model:v1 ./ai-model
docker build -t iris-frontend:v1 ./frontend
docker images
```

Confirm that iris-ai-model:v1 and iris-frontend:v1 appear in the image list.

5.1 Load the images into Minikube

```
minikube image load iris-ai-model:v1
minikube image load iris-frontend:v1
minikube image ls
```

The image list should include docker.io/library/iris-ai-model:v1 and docker.io/library/iris-frontend:v1.

Important: The working method for this setup is to build using the normal Docker Desktop daemon and then run `minikube image load`. The cluster uses containerd, so avoid relying on `eval "$(minikube docker-env)"` to build images.

6. Create Kubernetes YAML files

```
cd ~/docker-compose-ai/kubernetes
```

6.1 MongoDB: mongodb.yaml

```
nano mongodb.yaml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongodb
          image: mongo:7.0
          ports:
            - containerPort: 27017
          volumeMounts:
            - name: mongo-storage
              mountPath: /data/db
      volumes:
        - name: mongo-storage
          persistentVolumeClaim:
            claimName: mongo-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  selector:
    app: mongodb
  ports:
    - port: 27017
      targetPort: 27017
```

This creates a 1 GiB PersistentVolumeClaim, a MongoDB Deployment and a ClusterIP Service. The claim provides persistent storage for MongoDB data.

6.2 AI Deployment: ai-deployment.yaml

```
nano ai-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iris-ai-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: iris-ai
  template:
    metadata:
      labels:
        app: iris-ai
    spec:
      containers:
        - name: iris-ai-container
          image: iris-ai-model:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          env:
            - name: MONGO_URL
              value: "mongodb://mongodb:27017/"
```

replicas: 1 starts one AI Pod. The MongoDB URL uses the Kubernetes Service name mongodb, not localhost.

6.3 AI Service: ai-service.yaml

```
nano ai-service.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: iris-ai-service
spec:
  type: ClusterIP
  selector:
    app: iris-ai
  ports:
    - port: 5000
```

```
      targetPort: 5000
```

6.4 Frontend Deployment: frontend-deployment.yaml

```
nano frontend-deployment.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iris-frontend-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: iris-frontend
  template:
    metadata:
      labels:
        app: iris-frontend
    spec:
      containers:
        - name: iris-frontend-container
          image: iris-frontend:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
```

6.5 Frontend Service: frontend-service.yaml

```
nano frontend-service.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: iris-frontend-service
spec:
  type: NodePort
  selector:
    app: iris-frontend
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

The frontend Service uses NodePort 30080. For this WSL workflow, browser access is done with kubectl port-forward.

6.6 Verify all YAML files exist

```
ls -l
```

Expected: mongodb.yaml, ai-deployment.yaml, ai-service.yaml, frontend-deployment.yaml and frontend-service.yaml. If a file is missing, create and save it before applying.

7. Deploy the application

```
cd ~/docker-compose-ai/kubernetes
minikube kubectl -- apply -f mongodb.yaml
minikube kubectl -- apply -f ai-deployment.yaml
minikube kubectl -- apply -f ai-service.yaml
minikube kubectl -- apply -f frontend-deployment.yaml
minikube kubectl -- apply -f frontend-service.yaml
```

Successful commands report resources created, configured or unchanged.

7.1 Check Pods, Deployments and Services

```
minikube kubectl -- get pods
minikube kubectl -- get deployments
minikube kubectl -- get services
```

Wait for all Pods to show Running and READY 1/1:

Component	Expected Pods	Ready
MongoDB	1	1/1
AI model	1	1/1
Frontend	1	1/1

A Pod can show ContainerCreating briefly while its image is downloaded or its storage is mounted. Wait and run get pods again. If it remains stuck, inspect it with kubectl describe pod.

8. Access and test the application

8.1 Frontend port-forward — Ubuntu terminal 1

```
minikube kubectl -- port-forward service/iris-frontend-service 8080:80
```

Keep this terminal running.

8.2 AI API port-forward — Ubuntu terminal 2

```
minikube kubectl -- port-forward service/iris-ai-service 5000:5000
```

Keep this terminal running. Open a browser on Windows at **http://localhost:8080**. Use the default Iris measurements 5.1, 3.5, 1.4, 0.2 or enter your own four values. Click Predict Flower and then View History.

8.3 Test the AI API — Ubuntu terminal 3

```
curl http://localhost:5000/
```

```
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features":[5.1,3.5,1.4,0.2]}'
```

```
curl http://localhost:5000/history
```

The prediction response contains prediction, class_id and confidence. History returns saved prediction records.

9. Scale the AI Deployment

This is the main demonstration of the practical. A Deployment maintains the number of Pod replicas requested in its specification.

9.1 Check initial replica count

```
minikube kubectl -- get pods -l app=iris-ai
minikube kubectl -- get deployments
```

Initially there should be one AI Pod.

9.2 Scale from one to three replicas

```
minikube kubectl -- scale deployment iris-ai-deployment --replicas=3
minikube kubectl -- get pods -l app=iris-ai
minikube kubectl -- get deployments
```

Wait until three AI Pods show READY 1/1 and Running. The AI Deployment should show 3/3 ready replicas.

Explanation: Kubernetes creates two additional Pods using the same AI image. iris-ai-service selects every Pod labelled app: iris-ai and routes requests to available endpoints.

9.3 Scale down to one replica

```
minikube kubectl -- scale deployment iris-ai-deployment --replicas=1
minikube kubectl -- get pods -l app=iris-ai
minikube kubectl -- get deployments
```

Kubernetes removes the extra Pods and maintains one AI replica. This is manual scaling, not Horizontal Pod Autoscaling (HPA).

10. MongoDB records, logs and final checks

10.1 View MongoDB prediction history

```
minikube kubectl -- exec -it deployment/mongodb -- mongosh
```

```
show dbs
use iris_database
show collections
db.predictions.find().pretty()
exit
```

10.2 View logs

```
minikube kubectl -- logs deployment/iris-ai-deployment
minikube kubectl -- logs deployment/mongodb
minikube kubectl -- logs deployment/iris-frontend-deployment
```

10.3 Inspect all resources and storage

```
minikube kubectl -- get all
minikube kubectl -- get pvc
minikube kubectl -- get nodes
```

11. Stop and resume

Stop the two port-forward commands with Ctrl+C in their terminals.

```
minikube stop
```

This stops the cluster but retains its configuration and resources. To resume:

```
minikube start
minikube status
minikube kubectl -- get pods
```

> Avoid minikube delete if you want to keep the cluster. Avoid deleting the MongoDB PVC if you want to retain prediction history.

12. Troubleshooting — errors encountered and their solutions

12.1 Docker command not found in WSL

```
The command 'docker' could not be found in this WSL 2 distro.
```

Cause: Docker Desktop WSL integration was not enabled. **Fix:** Docker Desktop → Settings → Resources → WSL Integration → enable Ubuntu, apply, reopen Ubuntu, then run:

```
docker --version
docker ps
```

12.2 Minikube docker-env warning with containerd

```
Using the docker-env command with the containerd runtime is a highly experimental feature
```

Cause: Minikube uses containerd. The recommended workflow here is to build using Docker Desktop and transfer images with minikube image load. Do not depend on docker-env for building in this configuration.

12.3 BuildKit image pull / inspect returns 404

```
failed to inspect pulled image moby/buildkit:buildx-stable-1:
Error response from daemon: 404 page not found
```

Cause: Buildx attempted to start a BuildKit container and failed to retrieve or inspect its builder image. **Workaround used:** exit the Minikube docker environment, build with Docker Desktop from the project root, and load the images into Minikube:

```
eval "$(minikube docker-env -u)"
cd ~/docker-compose-ai
docker build -t iris-ai-model:v1 ./ai-model
docker build -t iris-frontend:v1 ./frontend
minikube image load iris-ai-model:v1
minikube image load iris-frontend:v1
```

12.4 Buildx default builder cannot be removed / desktop-linux protocol error

```
failed to remove default: context builder cannot be removed
additional instances of driver "docker" cannot be created
Cannot load builder desktop-linux: protocol not available
```

The default builder is associated with the default Docker context; it cannot be removed as an ordinary named builder. The Docker driver also does not support an additional independent builder for the same context. The desktop-linux error is separate. These errors do not require deleting Docker contexts to complete this practical.

12.5 Docker build says path ./ai-model not found

```
unable to prepare context: path "./ai-model" not found
```

The build was run from ~/docker-compose-ai/kubernetes. Move to the parent project folder before building:

```
cd ~/docker-compose-ai
docker build -t iris-ai-model:v1 ./ai-model
docker build -t iris-frontend:v1 ./frontend
```

12.6 YAML file does not exist

```
error: the path "mongodb.yaml" does not exist
```

The file had not been created in the kubernetes directory. Create it with nano mongodb.yaml, paste the complete YAML from Section 6.1, save, and confirm with ls.

12.7 apiVersion and kind are not set

```
error validating "ai-deployment.yaml":
[apiVersion not set, kind not set]
```

The YAML file was empty or the contents were not saved. Rewrite it using a heredoc to avoid editor/save mistakes:

```
cat > ai-deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: iris-ai-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: iris-ai
  template:
    metadata:
      labels:
        app: iris-ai
    spec:
      containers:
        - name: iris-ai-container
          image: iris-ai-model:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          env:
            - name: MONGO_URL
              value: "mongodb://mongodb:27017/"
EOF
```

If frontend-deployment.yaml has the same problem, recreate it using the complete configuration in Section 6.4.

12.8 MongoDB Pod initially shows ContainerCreating

This can be normal while the image is pulled and storage is mounted. Wait and run:

```
minikube kubectl -- get pods
```

If it remains stuck, inspect the Pod and PVC:

```
minikube kubectl -- describe pod <mongodb-pod-name>
minikube kubectl -- get pvc
```

Replace the placeholder with the actual MongoDB Pod name.

12.9 ImagePullBackOff or ErrImagePull

Check that the images are loaded and image names match the manifests:

```
minikube image ls
grep -n "image:" ai-deployment.yaml frontend-deployment.yaml
```

The manifests use imagePullPolicy: IfNotPresent so Kubernetes can use images already loaded into Minikube.

13. Examiner demonstration checklist

No.	Command / action	Demonstrates
1	minikube version	Minikube installation
2	minikube status	Cluster is running
3	minikube kubectl -- get nodes	Node is Ready
4	minikube kubectl -- get pods	All application Pods
5	minikube kubectl -- get deployments	Deployment status
6	minikube kubectl -- get services	Service networking
7	Open http://localhost:8080	Working frontend
8	Run curl prediction request	AI API response
9	Scale Deployment to 3 replicas	Scaling operation
10	get pods -l app=iris-ai	Three AI Pods
11	MongoDB mongosh query	Stored prediction history
12	Scale Deployment back to 1	Scale-down operation

Take screenshots of the Pods running, the browser prediction, the three AI replicas after scaling, and the MongoDB records.

14. Viva questions and answers

1. What is Kubernetes?
An open-source container orchestration platform used to deploy, manage and scale containerized applications.

2. What is Minikube?
A tool that runs a local Kubernetes cluster for development and learning.

3. What is a Pod?
The smallest deployable unit in Kubernetes. It contains one or more containers.

4. What is a Deployment?
A resource that manages Pods and maintains the requested number of replicas.

5. What is a Service?
A stable network endpoint that provides access to a set of Pods.

6. What is scaling?
Increasing or decreasing the number of application replicas.

7. What does replicas: 3 mean?
Kubernetes should maintain three Pods for that Deployment.

8. Docker vs Kubernetes?
Docker builds and runs containers; Kubernetes orchestrates containerized workloads in a cluster.

9. Docker Compose vs Kubernetes?
Compose defines and runs multi-container applications; Kubernetes manages applications through cluster resources such as Pods, Deployments and Services.

10. Why imagePullPolicy: IfNotPresent?
It allows Kubernetes to use a matching image already present on the node rather than always pulling it.

11. Why mongodb://mongodb:27017/?
mongodb is the Kubernetes Service name and is resolved by cluster DNS.

12. What is a PersistentVolumeClaim?
A request for persistent storage used by a workload, here MongoDB.

13. What does port-forward do?
It forwards a local port to a Pod or Service for local access.

14. Is this HPA?
No. This practical uses manual scaling with kubectl scale. HPA is a separate autoscaling feature.

15. Quick command reference

```
# Start and inspect
minikube start
minikube status
minikube kubectl -- get nodes
minikube kubectl -- get pods
minikube kubectl -- get deployments
minikube kubectl -- get services

# Access in separate terminals
minikube kubectl -- port-forward service/iris-frontend-service 8080:80
minikube kubectl -- port-forward service/iris-ai-service 5000:5000

# Scale AI deployment
minikube kubectl -- scale deployment iris-ai-deployment --replicas=3
minikube kubectl -- get pods -l app=iris-ai
minikube kubectl -- scale deployment iris-ai-deployment --replicas=1

# Stop and resume
minikube stop
minikube start
```

Practical is complete when: Minikube node is Ready; MongoDB, AI and frontend Pods run; the browser produces an Iris prediction; prediction history is stored in MongoDB; and the AI Deployment can be scaled from one to three replicas and back to one.

PRACTICAL 11 – UPDATED APPENDIX

Complete Frontend Source Code (Standalone From Scratch)

Use this appendix with the Practical 11 Minikube guide. It fills the missing file dependency: **frontend/index.html**. Create the file exactly as shown before building the frontend image.

1. Create the frontend file

```
cd ~/docker-compose-ai
mkdir -p frontend
cd frontend
nano index.html
```

2. Paste this complete HTML, CSS and JavaScript

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Iris Flower Prediction</title>
  <style>
    * { box-sizing: border-box; }
    body {
      margin: 0; min-height: 100vh; padding: 30px 16px;
      font-family: Arial, sans-serif; background: #f3f6fb; color: #172033;
      display: flex; justify-content: center; align-items: flex-start;
    }
    .container {
      width: 100%; max-width: 650px; background: white; padding: 28px;
      border-radius: 14px; box-shadow: 0 8px 28px rgba(20,40,80,.10);
    }
    h1 { margin-top: 0; color: #17365d; }
    .subtitle { color: #667085; margin-bottom: 24px; }
    .fields { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; }
    label { display: block; font-size: 14px; font-weight: 600; }
    input {
      width: 100%; margin-top: 7px; padding: 11px;
      border: 1px solid #cbd5e1; border-radius: 7px; font-size: 15px;
    }
    button {
      border: 0; border-radius: 7px; padding: 12px 17px;
      margin: 18px 8px 0 0; background: #1d5fa7; color: white;
      font-size: 14px; cursor: pointer;
    }
    button.secondary { background: #475467; }
    #result, #history {
      margin-top: 20px; padding: 14px; border-radius: 8px;
      background: #f1f5f9; overflow-wrap: anywhere;
    }
    pre { white-space: pre-wrap; }
    @media (max-width: 480px) {
      .fields { grid-template-columns: 1fr; }
      .container { padding: 20px; }
    }
  </style>
</head>
<body>
  <main class="container">
    <h1>Iris Flower Prediction</h1>
    <p class="subtitle">Enter four flower measurements to predict the Iris species.</p>

    <div class="fields">
      <label>Sepal length (cm)
        <input id="f1" type="number" step="any" value="5.1">
      </label>
      <label>Sepal width (cm)
        <input id="f2" type="number" step="any" value="3.5">
      </label>
      <label>Petal length (cm)
        <input id="f3" type="number" step="any" value="1.4">
      </label>
      <label>Petal width (cm)
        <input id="f4" type="number" step="any" value="0.2">
      </label>
    </div>

    <button onclick="predictFlower()">Predict Flower</button>
    <button class="secondary" onclick="viewHistory()">View History</button>
    <section id="result" aria-live="polite">Prediction result will appear here.</section>
    <section id="history" aria-live="polite">Prediction history will appear here.</section>
  </main>

  <script>
```

```
    // When using Minikube, keep the AI service port-forward running:
    // minikube kubectl -- port-forward service/iris-ai-service 5000:5000
    const API_URL = "http://localhost:5000";

    async function predictFlower() {
      const ids = ["f1", "f2", "f3", "f4"];
      const features = ids.map(id => Number(document.getElementById(id).value));
      const resultBox = document.getElementById("result");

      if (features.some(value => !Number.isFinite(value))) {
        resultBox.textContent = "Please enter valid numeric values in all four fields.";
        return;
      }

      resultBox.textContent = "Predicting...";
      try {
        const response = await fetch(`${API_URL}/predict`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ features })
        });
        const data = await response.json();
        if (!response.ok) throw new Error(data.error || "Prediction request failed");
        resultBox.textContent =
          `Predicted species: ${data.prediction} | Class ID: ${data.class_id} | Confidence: ${data.confidence}`;
      } catch (error) {
        resultBox.textContent = `Error: ${error.message}. Check that the AI API port-forward is running.`;
      }
    }

    async function viewHistory() {
      const historyBox = document.getElementById("history");
      historyBox.textContent = "Loading history...";
      try {
        const response = await fetch(`${API_URL}/history`);
        const records = await response.json();
        if (!response.ok) throw new Error("Could not load history");
        historyBox.innerHTML = "";
        if (!records.length) {
          historyBox.textContent = "No predictions have been saved yet.";
          return;
        }
        const pre = document.createElement("pre");
        pre.textContent = records.map((item, index) =>
          `${index + 1}. ${item.prediction} | confidence: ${item.confidence} | features: ${item.features.join(", ")}`
        ).join("\\n");
        historyBox.appendChild(pre);
      } catch (error) {
        historyBox.textContent = `Error: ${error.message}. Check the AI API and MongoDB Pod.`;
      }
    }
  </script>
</body>
</html>
```

3. Save the file and create the frontend Dockerfile

```
Press Ctrl+O, Enter, Ctrl+X to save nano.

nano Dockerfile
```

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

4. Build and load the frontend image

```
cd ~/docker-compose-ai
docker build -t iris-frontend:v1 ./frontend
minikube image load iris-frontend:v1
minikube image ls
```

5. Run both port-forwards in separate Ubuntu terminals

```
Terminal 1:
minikube kubectl -- port-forward service/iris-frontend-service 8080:80

Terminal 2:
minikube kubectl -- port-forward service/iris-ai-service 5000:5000
```

Open http://localhost:8080 in the browser. Keep both terminal sessions running while testing. The frontend calls the AI API at http://localhost:5000, which is provided by the second port-forward.

Standalone exam reminder

If Practical 11 is assigned independently, begin with the Minikube installation and cluster setup in the main guide, create the project folders, create the AI files from the main guide, create this frontend file, build and load both images, then create and apply all five Kubernetes YAML resources. Do not assume the Practical 10 folder already exists.